Lets AI - OpenCode

Yogesh Haribhau Kulkarni

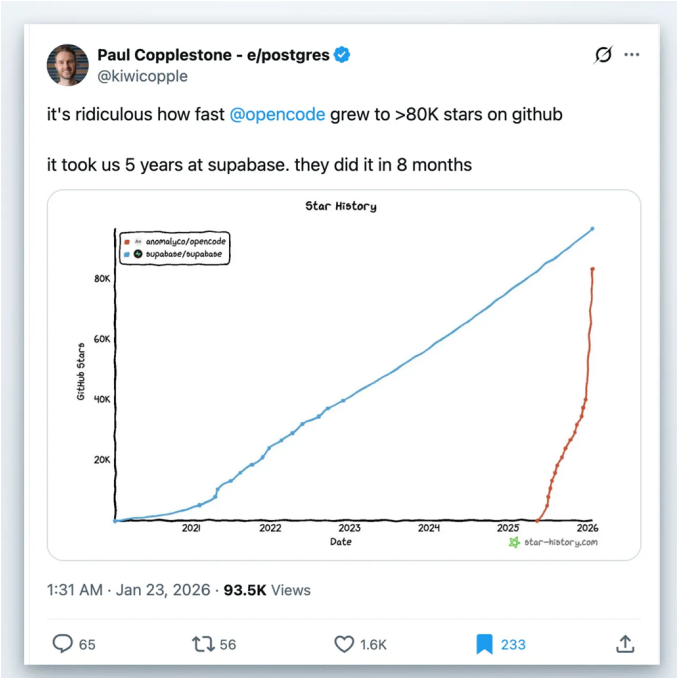
YHK

OpenCode

YHK

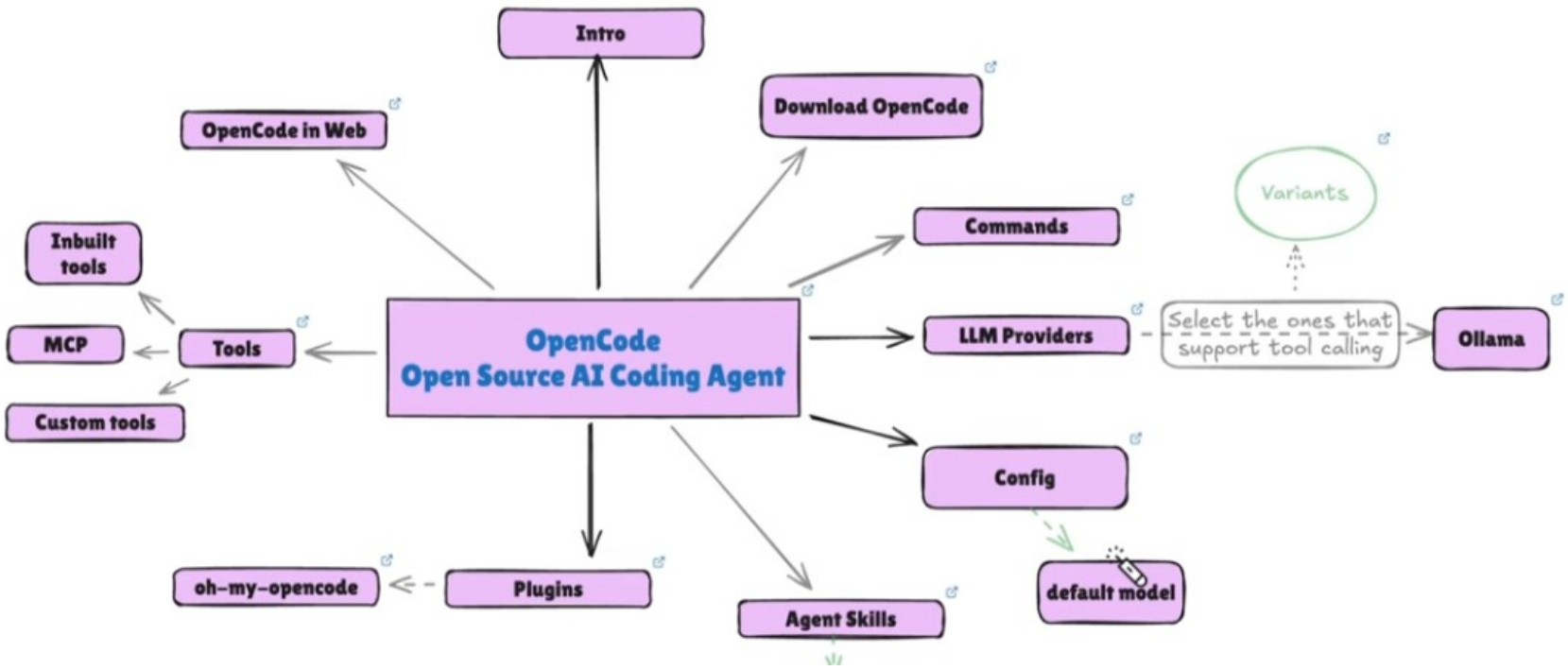
OpenCode: Rapid Growth

- **90K+ GitHub stars** accumulated in less than a year: one of the fastest-growing developer tools in history.
- **650,000+ monthly users** as of early 2026.
- **Official partnerships:** OpenAI and GitHub have formally partnered with OpenCode.
- Born as a Claude Code alternative; evolved into a model-agnostic agent arguably more capable than what it was imitating.
- Even the creator of Supabase publicly praised its momentum.
- Developers who were paying $20/month for Claude Code realized they could use any model from any provider, from their terminal, without restrictions.



(Ref: JP Caparas, "The definitive guide to OpenCode: from first install to production workflows", Dev Genius / reading.sh, Feb 2026)

OpenCode Ecosystem



(Ref: OpenCode CRASH Course - Data Science Basics)

What is OpenCode?

- Open-source coding agent built for the terminal (TUI).
- **IDE Integration possible**: Works with VS Code, Cursor, and any terminal-supporting IDE
- **GitHub Integration**: Automate issues and PRs directly from comments
- **Model-Agnostic:** Supports 75+ AI models (OpenAI, Claude, Groq, Ollama, and more).
  - *Local:* Ollama (fully offline, zero data egress)
  - *Cloud:* OpenAI, Claude, Groq, Gemini, etc.
- **Privacy-Focused:** Runs locally or uses your own API keys, your code stays on your machine.
- **Built With:** Rust/Go for performance; Bubble Tea for the TUI layer.

YHK

Capabilities

Think of it as having a senior developer sitting next to you

- Writing and debugging code
- Understanding existing codebases
- Refactoring and optimization
- Answering technical questions
- Automating repetitive tasks

YHK

Key Features

- **Interactive TUI:** Smooth terminal UI built with Bubble Tea; keyboard-driven workflow.
- **Contextual Understanding:** Reads your entire repo, understands dependencies, structure, languages.
- **Agentic Workflows:** Plans changes, searches files, executes shell commands, modifies code autonomously.
- **Plan / Build Modes/Agents:** Review proposed changes before applying, deterministic control.
- **Slash Commands:** `/init`, `/help`, `/models`, `/share`, and more.
- **Context Mentions:** `@file:`, `@workspace` for scoped AI queries.

YHK

Core Integration

- **LSP Integration:** Language Server Protocol for real-time code intelligence (completions, diagnostics, go-to-def).
- **MCP (Model Context Protocol):** Plug in external tools and data sources to extend agent capabilities.
- **ACP Support:** Agent Communication Protocol for inter-agent coordination.
- **Agent Skills & Custom Tools:** Define reusable agent behaviors and wire in custom shell/API tools.
- **Rules & Permissions:** Fine-grained control over what the agent can read, write, or execute.
- **Multi-Provider Routing:** Failover and cost-based routing across model providers.

YHK

OpenCode vs Other Tools

Feature	OpenCode	Claude Code	Cursor
Open Source	✅	❌	❌
Free Tier	✅	Limited	Limited
Terminal-first	✅	✅	❌
Provider Agnostic	✅	❌ (Claude only)	❌
Local Models	✅	❌	✅
IDE Integration	✅	✅	Native

(Ref: OpenCode - nxcode.io

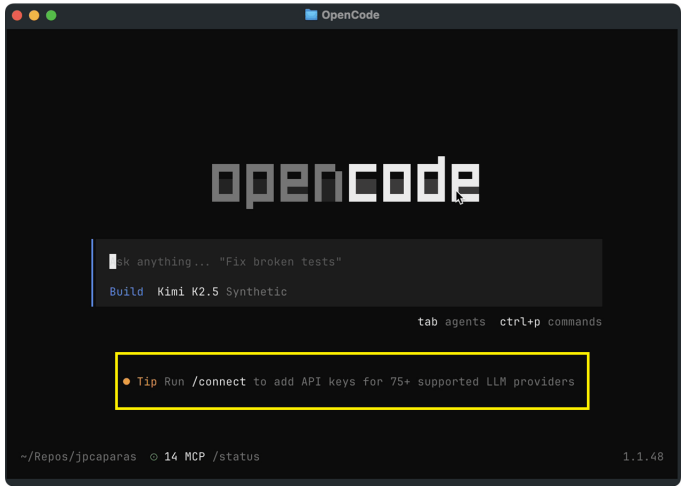
Installation: All Platforms

- **Desktop App:** Download directly from opencode.ai/download for a native desktop experience.
- **Quick Install (Linux/macOS):**
  `$ curl -fsSL https://opencode.ai/install | sh`
- **npm/bun:** `npm i -g opencode-ai@latest` or `bun add -g opencode-ai`
- **macOS (Homebrew):** `brew install sst/tap/opencode`
- **Windows (Chocolatey):** `choco install opencode`
- **Windows (Scoop):** `scoop install opencode`
- **Arch Linux (paru):** `paru -S opencode-bin`
- Verify installation: `opencode --version`
- If installed using npm on Windows 11 then
  - Installation directory:
    `%USERPROFILE%\AppData\Roaming\npm\node_modules\opencode-ai\`
  - Configuration and data (session data, logs, and authentication tokens):
    `%USERPROFILE%\.local\share\opencode\auth.json`
  - Global plugins: `%USERPROFILE%\.config\opencode\plugins\`
  - Global configuration file for user-wide settings (providers, models, etc.):
    `%USERPROFILE%\.config\opencode\opencode.json`

YHK

Getting Started: Connecting a Provider

- **Launch:** Run `opencode` in any project directory. The TUI loads; your first step is connecting a provider.
- **Connect:** Type `/connect` → interactive menu → select provider → paste API key.
- **Bring Your Own Keys:** Set `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, or point to Ollama.
- **Configure Provider:** Global config `~/.config/opencode/opencode.json` or project-specific `./opencode.json`
- **Config Format:** JSON-based `opencode.json`; supports project-level and global overrides.
- **First Step After Connect:** Run `/init` to ground the agent in your project: scans the codebase and creates `AGENTS.md`.



(Ref: JP Caparas, "The definitive guide to OpenCode: from first install to production workflows", Dev Genius / reading.sh, Feb 2026)

YHK

Providers in OpenCode

OpenCode supports 75+ LLM providers. Here are the most useful options.

OpenCode Zen (Curated Gateway)

- Pre-benchmarked models with optimized configurations
- No setup headaches — models just work
- Access to free models (GLM 4.7 Free, Kimi K2.5 Free, Big Pickle)
- Competitive pricing (often cheaper than direct APIs)
- Team management features built-in

Pricing (per 1M tokens):

- Claude Sonnet 4.5: $3.00 input / $15.00 output
- Qwen3 Coder 480B: $0.45 input / $1.50 output
- Free models: $0 (may train on user data)

Setup:

- Sign in at `opencode.ai/auth`
- Add billing details
- Run `/connect`, select OpenCode, paste your key

YHK

Custom OpenAI-Compatible Providers

Connection Steps:

- Run `/connect`
- Select `Other`
- Enter provider ID
- Paste your API key

Use any provider with an OpenAI-compatible API.

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "my-provider": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "My Custom Provider",
      "options": {
        "baseURL": "https://api.my-provider.com/v1"
      },
      "models": {
        "my-model": {
          "name": "My Model" }}}}}
```

YHK

Plan vs Build Mode

Switch between them using the Tab key.
Plan Mode:

- Read-only - Cannot make changes to your code
- Analyzes and explores your codebase
- Suggests implementation strategies
- Perfect for understanding before modifying

Build Mode (Default)

- Full access - Can read, write, and modify files
- Executes code changes
- Runs commands
- Best for actual development work

Pro Tip: Always start complex features in Plan mode to outline your approach, then switch to Build mode for implementation.

YHK

Context Mentions & Image Support

- **`@file:path`** — focus the agent on a specific file:

  ```
  @file:main.go fix the bug where user input isn't sanitized
  ```

- **`@workspace`** — ask about the broader codebase:

  ```
  Show me how API routes are structured
  ```

- **Onboarding power:** Instead of manually `grep`-ing and reading files, just ask. Especially useful when exploring an unfamiliar codebase.
- **Image drag-and-drop:** Drag and drop screenshots or mockups directly into the terminal. OpenCode scans them and includes them in context — useful for implementing UI from a design mockup.
- **Tab to Plan mode:** For complex changes, use `Tab` to switch to Plan mode, describe the feature, and review the proposed approach before any code is written.

YHK

Session Management

- `/sessions` (aliases: `/resume`, `/continue`) — opens a session picker showing recent conversations; select one to resume with full context.
- **CLI continue flags:**
  - `opencode --continue` or `opencode -c` — resumes the most recent session.
  - `opencode session list` — lists all sessions with their IDs.
  - `opencode session continue <id>` — resumes a specific session by ID.
- **Session titles:** Auto-generated from your first prompt. Name them upfront for easy retrieval: `opencode run --title "Payment API refactor" "Refactor the payment processing module"`
- `/share` — generates a shareable URL for async code review, debugging docs, or team collaboration.
- `/compact` (alias: `/summarize`) — compresses the current session context to save tokens while preserving key decisions.

(Ref: JP Caparas, "The definitive guide to OpenCode", Dev Genius / reading.sh, Feb 2026)

YHK

Undo, Redo & Non-Interactive Mode

- `/undo` — reverts file changes from the last agent response; restores original prompt so you can rephrase and retry.
- `/redo` — re-applies changes that were undone.
- **Non-interactive mode** (`opencode run`) — pipe prompts for scripting and CI:
- **GitHub Action:** Use `opencode run` in CI pipelines — e.g., auto-respond to PR comments, generate changelogs, run AI-assisted linting.

```
# One-shot task
opencode run "Add JSDoc comments to all exported functions in src/"

# With title for easy retrieval
opencode run --title "Add JSDoc" "Add JSDoc to src/"

# Continue last session non-interactively
opencode -c run "Now add unit tests for those functions"
```

YHK

Configuration

Overview

- OpenCode is fully configurable at **two scopes**: Global (`~/.config/opencode/`) and Project (`.opencode/` in repo root).
- **Config priority** (highest → lowest): CLI flags → Custom env (`OPENCODE_CONFIG_DIR`) → Project `opencode.json` → Global `opencode.json` → Remote org defaults (`.well-known/opencode`).
- **Subdirectories** (plural names preferred): `agents/`, `skills/`, `tools/`, `modes/`, `commands/`, `themes/`, `plugins/`.
- **Schema-validated:** Config is validated against `opencode.ai/config.json`: editors autocomplete fields automatically.
- **Claude Code compatibility:** Falls back to `CLAUDE.md` / `~/.claude/` conventions if OpenCode files are absent.
- Each section below covers: what ships built-in → what you can customize → how to create your own.

The core problem: Context is a finite budget. Dumping everything into one file degrades quality: the agent buries critical constraints under noise. OpenCode solves this with *tiered, purposeful* context loading.

YHK

Customization Layers: The Mental Model

- **AGENTS.md: "The Onboarding Doc":** *Always-on*, repo-committed, team-shared. Tells *any* agent how this project works: stack, build commands, test steps, conventions. Think of it as the README for AI: written once, applied everywhere.
- **Rules: "Standing Orders":** *Always-on*, injected into every session. Global rules (`~/.config/opencode/AGENTS.md`) encode personal preferences never shared with the team. Multi-file rules via `opencode.json` allow conditional loading per glob/path. Same intent as AGENTS.md: just scoped differently (personal vs. project vs. package).
- OpenCode has three extension mechanisms: commands, skills, and agents. Commands are what to do. Skills are how to do it. Agents are who does it.
-
- **Commands: "The shortcuts for repetitive prompts":** stored in `Create .opencode/commands/test.md` and triggered with /
- **Skills: "The On-Call Expert":** *Lazy-loaded*: Unlike commands (which execute), skills inform. The agent reads only the name + description at startup, then fetches full content only when the task is relevant. Prevents token waste on instructions that don't apply to the current task. Think: nobody memorizes the entire wiki before starting a ticket.
- **Agents: "The Specialist Persona":** Own model, temperature, tool set,

YHK

Customization Layers: When to Use What

The layering principle: Rules = what to always know. Skills = what to know when needed. Agents = who is doing the work and with what capabilities.

- **Use AGENTS.md when:** The instruction is *project-wide*, stable, and useful to every agent, every session: build steps, test commands, folder conventions, PR rules. Commit it to Git; share with the team. Auto-generate with `/init`.
- **Use Rules when:** The instruction is always relevant but *not* project-specific: personal tone, preferred libraries, output format, privacy rules. Global rules stay on your machine, never in the repo.
- **Use Skills when:** The instruction is *task-specific* and only occasionally needed: e.g., a `git-release` skill for changelogs, a `db-migration` skill for schema changes. Keeps context lean; agent loads it only when it matches the task.
- **Use Custom Agents when:** You need a *different set of capabilities* for a role: a `Review` agent that can't write files, a `Debug` agent with only bash + read, a `Security` agent using a different model.
- **Use Commands (/) when:** You have a *repetitive prompt*: a parameterized template you type often. Commands invoke agents, pass arguments, and save keystrokes.

YHK

Customization Layers: Summary Table

Layer	Loaded	Scope
AGENTS.md	Always	Project / Package
Rules	Always	Personal / Team
Skills	On-demand	Any
Agents	When invoked	Session role
Commands	When typed	Shortcut

YHK

AGENTS.md

AGENTS.md helps OpenCode understand: project structure, Coding conventions, Technology stack etc

```
## Project: My App

## Tech Stack
- Next.js 14 with App Router
:

## Conventions
- Use functional components
:

## Structure
- /app - Next.js app router pages
- /components - Reusable UI components
:
```

YHK

Rules: Persistent AI Instructions

- **What:** Plain-text instructions injected into every LLM context. Think of them as standing orders: coding style, conventions, constraints.
- **Built-in bootstrap:** `/init` auto-generates `AGENTS.md` by scanning your repo: languages, deps, folder structure, patterns.
- **Scope:**
  - *Project rules*: `./AGENTS.md`: commit to Git; shared across team.
  - *Global rules*: `~/.config/opencode/AGENTS.md`: personal, not committed.
  - *Claude Code fallbacks*: `~/.claude/CLAUDE.md` if no `AGENTS.md`
- **Multi-file rules** via `opencode.json`: glob patterns, remote URLs supported.
- **Lazy loading:** Reference `@rules/style.md` in `AGENTS.md`: agent reads files on-demand, not upfront (saves tokens).
- **Create custom:** Write any Markdown file → list it under `instructions` in `opencode.json`. No special syntax: plain instructions work.

```
{ "instructions": ["docs/standards.md",
    "test/guidelines.md", "packages/*/AGENTS.md",
    "https://myorg.com/team-rules.md"] }
```

YHK

Agents: Built-in & Custom

- **Two types:** *Primary agents* (you interact with directly, cycle via `Tab`) and *Subagents* (specialists invoked by primary agents or `@mention`).
- **Built-in Primary Agents:**
  - `Build`: default; all tools enabled; full file + shell access.
  - `Plan`: read-only; all mutating tools set to `ask`; safe for analysis.
- **Built-in Subagents:**
  - `General`: full tool access; multi-step parallel tasks.
  - `Explore`: read-only codebase navigator; fast file/pattern search.
- **Common custom agents:** `Review` (read-only + docs), `Debug` (bash + read), `Docs` (write, no shell), `Security` (audit mode).
- **Create custom:** Run `opencode agents create` → CLI prompts for scope, description, tools, model → writes `.opencode/agents/<name>.md`.
- **Agent frontmatter** fields: `model`, `temperature`, `mode` (`primary`/`subagent`/`all`), `tools`, `permission`, `steps` (max iterations), `color`.
- **Set default agent** in `opencode.json`: `"default_agent": "build"`: applies across TUI, CLI, IDE, GitHub Action.

YHK

Tools: Built-in & Custom

- **What:** Functions the LLM calls to act on your codebase. All built-in tools are enabled by default.
- **Some Built-in Tools:**
  - `bash`: execute shell commands (`git`, `npm`, `make`, etc.)
  - `write`: create/overwrite files
  - `ls`: list directory contents
  - `skill`: load a `SKILL.md` on-demand
  - `todo`: manage task lists for multi-step work
- **Custom Tools:** TypeScript/JS files in `.opencode/tools/` (project) or `~/.config/opencode/tools/` (global). Filename = tool name. Can wrap scripts in any language (Python, Bash, Go, etc.).
- **Override built-ins:** Custom tool with same name replaces the built-in (e.g., `bash.ts` → sandboxed shell).

```
// .opencode/tools/db-query.ts
import { tool } from "@opencode-ai/plugin"
export default tool({ description: "Run SQL query",
  args: { query: tool.schema.string() },
  async execute(args) { /* call your DB */ }
})
```

YHK

Agent Skills: Reusable Behavior Packages

- **What:** Markdown instruction bundles loaded on-demand by agents via the `skill` tool. Unlike rules (always loaded), skills are lazy: the agent picks them when relevant.
- **No built-in skills ship with OpenCode**: skills are community/team-created. Claude Code-compatible skills from `~/.claude/skills/` work as fallbacks.
- **Discovery locations** (all checked at startup):
  - `.opencode/skills/<name>/SKILL.md`: project-local
  - `~/.config/opencode/skills/<name>/SKILL.md`: global
  - `.claude/skills/` and `.agents/skills/`: compatibility paths
- **Name rules:** Lowercase alphanumeric + single hyphens; 1–64 chars; must match directory name.
- **Permissions:** Control skill access per agent with `allow` / `deny` / `ask` and wildcard patterns (`internal-*`).

YHK

Agent Skills: Create a custom skill

```
# .opencode/skills/git-release/SKILL.md
---
name: git-release
description: Create consistent releases and changelogs
license: MIT
compatibility: opencode
---
## What I do
- Draft release notes from merged PRs
- Propose a version bump
- Generate a ‘gh release create‘ command
## When to use me
Use when preparing a tagged release.
```

Permissions: Granular Action Control

- **What:** Per-tool access policy: `allow` (auto-run), `ask` (prompt user), or `deny` (block). Defaults: most tools `allow`; `doom_loop` and `external_directory` default to `ask`; `.env` files denied by default.
- **Scope:** Set globally in `opencode.json`; override per-agent in agent frontmatter. Agent rules take precedence.
- **Skill permissions:** grant/deny specific skills by name or pattern.
- **Wildcard global policy + overrides** (last matching rule wins):

```
{ "permission": {
    "*": "ask",          // ask for everything by default
    "bash": {
      "*": "ask",
      "git *": "allow",  // git commands auto-approved
      "npm *": "allow",
      "rm *": "deny"     // destructive: always blocked
    },
    "mymcp_*": "ask"   // all MCP tool calls: prompt first
  }
}
```

YHK

Modes: Contextual Tool Profiles

- **What:** Named configurations combining a system prompt + tool subset + temperature. Switch modes to change agent behavior without switching agents.
- **Difference from Agents:** Agents are full AI configurations with their own identity and model; Modes are overlays on top of an agent.
- **Built-in Modes:**
  - `Build` (default): all tools on, full write access
  - `Plan`: analysis only; mutating tools set to `ask`
- **Useful custom modes:** `Review` (read-only + docs), `Debug` (bash + read, no writes), `Refactor` (edit only, no shell), `Docs` (write files, no system commands).
- **Create custom mode**: Markdown file in `.opencode/modes/` or `~/.config/opencode/modes/`:
- **Switch modes** in TUI: `Tab` key cycles between `Build` and `Plan` by default.

YHK

Modes: Custom

```
---
name: debug
description: Investigate issues, no file changes
tools:
  bash: true
  read: true
  grep: true
  write: false
  edit: false
temperature: 0.2
---
You are in debug mode. Diagnose and report only.
Do NOT modify any files. Use bash and read tools
to investigate. Output a structured findings report.
```

Commands: Slash-Invokable Shortcuts

- **What:** Reusable prompt templates invoked with / in the TUI. Eliminate repetitive prompt writing for common workflows.
- **Built-in commands:**
  - `/init`: scan project, generate `AGENTS.md`
  - `/models`: browse and switch active model
  - `/help`: list all available commands
- **Create custom commands** in `opencode.json` (or as `.opencode/commands/<name>.md` files):
- **`$ARGUMENTS`:** Anything typed after `/command-name` is injected here: makes commands parameterized.
- **Team sharing:** Commit `.opencode/commands/` to Git: team gets it.

```
{ "command": {
    "test-report": {
      "description": "Full test suite with coverage",
      "template": "Run test suite with coverage report. Suggest fixes."
    },
  }
}
```

YHK

MCP Servers: External Tool Integration

- **What:** Model Context Protocol (MCP) servers expose external tools and data sources to OpenCode. Any MCP-compatible server becomes a tool the agent can call natively.
- **Why:** Extends OpenCode beyond the filesystem: databases, APIs, cloud services, monitoring tools, internal systems.
- **Popular MCP servers:** GitHub, GitLab, Slack, PostgreSQL, Puppeteer, Brave Search, Filesystem (extended), AWS, Docker, Linear, Jira, Notion, Sentry.
- **Configure in `opencode.json`:**
- **Tool naming:** MCP tools are prefixed by server name: `github_create_issue`, `postgres_query`, etc. Use `mymcp_*` in permissions to control all tools from one server.
- **Org defaults:** Deploy company-wide MCP servers via `.well-known/opencode`: auto-loaded for all authenticated users.
- **Build your own:** Any HTTP/stdio process implementing the MCP spec becomes a server: SDKs available in TypeScript, Python, Go.

YHK

MCP Servers: External Tool Integration

```
{ "mcp": {
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "$GITHUB_TOKEN" }
    },
    "postgres": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres",
               "postgresql://localhost/mydb"]
    }
  }
}
```

Custom Tools: Extending the Agent's Hands

- **What:** TypeScript/JS files that define new functions the LLM can call. Can wrap any language internally (Python, Bash, Rust, etc.).
- **Placement:** `.opencode/tools/<name>.ts` (project) or `~/.config/opencode/tools/<name>.ts` (global).
- **One file = one or more tools:** Single default export → tool named after file. Multiple named exports → tools named `<file>_<export>`.
- **Multi-tool file example:**
- **Context available:** `context.agent`, `context.session`, `context.directory` (working dir), `context.worktree` (git root).
- **Override built-ins:** Same filename as a built-in (e.g., `bash.ts`) replaces the built-in completely: useful for sandboxing or auditing.
- **Any language via shell:** Call Python/Bash scripts inside `execute()` using `child_process`: tool definition is TS, implementation is anything.
- **Schema validation:** Use `tool.schema` (Zod) for type-safe arg definitions with `.describe()` for LLM-facing documentation.

YHK

Custom Tools: Extending the Agent's Hands

```
// .opencode/tools/math.ts
import { tool } from "@opencode-ai/plugin"
export const add = tool({ description: "Add two numbers",
  args: { a: tool.schema.number(), b: tool.schema.number() },
  async execute({ a, b }) { return a + b }
})
export const multiply = tool({ description: "Multiply",
  args: { a: tool.schema.number(), b: tool.schema.number() },
  async execute({ a, b }) { return a * b }
})
// creates: math_add, math_multiply
```

Models: Provider & Model Configuration

- **75+ models supported** across: Anthropic, OpenAI, Google Gemini, Groq, Mistral, Cohere, xAI, DeepSeek, Qwen, Ollama (local), Hugging Face, AWS Bedrock, Azure, Vertex AI, and more.
- Chose models with long context window and have access to tools (function calling)
- **Switch models** in TUI: `/models` command → browse and select. Or `-m provider/model` CLI flag at launch.
- **Configure in `opencode.json`:**
- **Variants:** Define multiple configurations of the same model (different temperature, max_tokens, reasoning effort) and cycle with `variant_cycle` keybind.
- **Per-agent model:** Each custom agent can specify its own model: e.g., `Plan` uses a cheap fast model; `Build` uses full Sonnet/Opus.
- **Small model for meta-tasks:** OpenCode uses a lightweight model (default: `gpt-5-nano` via Zen) for session titling and other housekeeping: configurable separately.
- **Authenticate:** `/connect` in TUI → select provider → paste API key. Supports OAuth for GitHub/GitLab providers.

YHK

Models: Provider & Model Configuration

```
{ "model": "anthropic/claude-sonnet-4-6",
  "provider": {
    "anthropic": { "apiKey": "$ANTHROPIC_API_KEY" },
    "openai":    { "apiKey": "$OPENAI_API_KEY" },
    "ollama":    { "url": "http://localhost:11434" }
  }
}
```

YHK

Real-World Use Cases

- **Understanding a Codebase:**
  `You: Explain the authentication flow in this project` OpenCode will analyze your files and provide a detailed explanation of how authentication works.
- **Adding a Feature:** `You: Add a dark mode toggle to the settings page` OpenCode will: Find the settings page, Identify the theme system, Implement the toggle, Update necessary styles
- **Debugging:**
  `You: The login form is not submitting. Here's the error: [paste error]` OpenCode will trace the issue and suggest fixes.
- **Multi-File Refactoring:** Rename, restructure, or migrate patterns across the entire codebase atomically.
- **Documentation Generation:** Auto-generate README, API docs, and inline comments from code context.
- **Test Generation:** Scaffold unit/integration tests with awareness of existing test patterns.
- **Security Auditing:** Identify unsanitized inputs, SQL injection points, or insecure defaults.

YHK

Demo

Initialize & Understand

- **Goal:** Show how OpenCode scans and understands a new project from scratch.
- **Setup:** Open terminal inside any Git repository.
- **Launch:** `$ opencode`
- **Command:** `> /init`
- **What Happens:** OpenCode scans all local files, infers stack (languages, deps, structure), and writes `AGENTS.md`, the AI's instruction manual for this project.
- **Key Point:** Every subsequent interaction is grounded in this context, no hallucinated file paths.

YHK

Fix & Refactor

- **Goal:** Demonstrate AI-driven code modification on a real file.
- **Command:** `> @file:main.go fix the bug where user input isn't sanitized`
- **Agent Action:** Reads `main.go`, identifies the unsanitized input vector, proposes a targeted patch.
- **Review Flow:**
  - Press `Tab` → switch to **Plan** mode: inspect the proposed diff before anything touches disk.
  - Press `Tab` → switch to **Build** mode: apply the change.
- **Key Point:** You stay in control, the agent proposes, you decide.

YHK

Tips

Tips for Beginners

- Be Specific: Don't say "Make it better" but say, "Refactor the handleSubmit function to use async/await and add error handling"
- Provide Context: Don't say "Fix the bug" but say, "The login form throws 'undefined is not a function' when clicking submit. The error occurs in auth.ts line 45"
- Use Plan Mode First: For complex features, ask OpenCode to create a plan before implementation.
- Leverage Image Support: Drag and drop screenshots or mockups into the terminal—OpenCode can understand images!
- Create Project-Specific Configs: Different projects may need different settings. Use local opencode.json files.

YHK

1. Root Rules File is Execution Memory

- Your `OPENCODE.md` (or workspace rules file) should define:
  - WHY — product purpose
  - WHAT — repo map
  - HOW — non-negotiable rules
- Keep it short and authoritative.
- If it becomes a knowledge dump, OpenCode will ignore critical constraints.
- Clear, structural guidance beats long explanations.

YHK

2. Turn Repeated Workflows into Templates

- If you repeat instructions twice, template them.
- Create reusable folders such as:

```
.opencode/tasks/
.opencode/templates/
```

- Standardize code review, refactoring, and release workflows.
- Prevent prompt drift across sessions and teammates.

3. Use Automation for Deterministic Guardrails

- Memory is probabilistic. Tooling is deterministic.
- Use project-level commands, pre-commit hooks, and CI checks.
- Examples:
  - Auto-run formatters
  - Run tests after core changes
  - Block unsafe directory edits (auth, migrations)
- Deterministic rules belong in tooling, not prompts.

YHK

4. Use docs/ for Progressive Disclosure

- Keep architecture, ADRs, and runbooks in `docs/`.
- Reference documentation — do not paste it into context.

```
docs/
  architecture.md
  decisions/
  runbooks/
```

- Smaller context improves reliability.

YHK

5. Add Local Rule Files Near Sharp Edges

- Place rule files near high-risk directories.
- Document invariants, constraints, and warnings.

```
src/auth/RULES.md
src/persistence/RULES.md
infra/RULES.md
```

- Context is loaded exactly where it matters.

6. Structure for Machine Onboarding

- If a new engineer would be confused, OpenCode will be too.
- Use clear boundaries:
  - `src/`
  - `api/`
  - `domain/`
  - `infra/`
- Keep naming consistent and nesting shallow.

YHK

7. Smaller Context Beats Larger Context

- More files in context does not mean better output.
- Large context dilutes signal and increases hallucination risk.
- Provide:
  - Exact files
  - Exact constraints
  - Exact task definition
- Precision beats volume.

YHK

8. Document Architecture Decisions (ADRs)

- OpenCode cannot infer historical decisions.
- Document why technologies and boundaries were chosen.

```
docs/decisions/
```

- Prevent the AI from "optimizing" your architecture away.

9. Treat Prompts as Versioned Assets

- Store reusable prompts in your repository.

```
tools/prompts/
feature-implementation.md
refactor-checklist.md
bug-triage.md
```

- Version and improve them over time.
- Avoid copy-paste drift across sessions.

YHK

10. Structure ¿ Prompting Skill

- Strong structure transforms OpenCode into a workspace-native engineer.
- Combine:
  - File-based rules
  - Workflow templates
  - Automated guardrails
  - Documented architecture
  - Minimal scoped context
- Prompts are temporary. Structure is permanent.

YHK

Summary

Why Use OpenCode?

- **Privacy-First:** Code never leaves your machine by default; no telemetry, no vendor data retention.
- **Cost Efficiency:** No subscription fees, pay only for actual API token usage; local models are free.
- **No Vendor Lock-in:** Switch AI providers in seconds via config; one tool, all models.
- **Terminal-Native Speed:** Faster context-switch than IDE plugins; integrates directly into your existing shell workflow.
- **Transparent & Auditable:** Fully open-source (Apache 2.0), inspect, fork, or extend anything.
- **Enterprise-Ready:** Supports self-hosted LLMs and air-gapped (physically isolated from all other networks) environments.
- **Bottom Line:** Faster dev loop, zero lock-in, full code privacy, AI that works *with* your terminal.

YHK

Quick Reference: Config Cheat Sheet

- **File locations** (P = project .opencode/, G = global ~/.config/opencode/):
  - Rules: P: AGENTS.md · G: AGENTS.md
  - Agents: P: agents/<name>.md · G: agents/<name>.md
  - Skills: P: skills/<name>/SKILL.md · G: skills/<name>/SKILL.md
  - Tools: P: tools/<name>.ts · G: tools/<name>.ts
  - Modes: P: modes/<name>.md · G: modes/<name>.md
  - Commands: opencode.json → "command": {...}
  - MCP: opencode.json → "mcp": {...}
  - Permissions: opencode.json → "permission": {...}
- **Key CLI commands:**
  - opencode agents create: scaffold a new agent interactively
  - /init: generate AGENTS.md for current project
  - /models: browse and switch models
  - /connect: authenticate a new provider
  - /share: share current session URL
- **Priority order** (highest → lowest): CLI flags → OPENCODE_CONFIG_DIR → .opencode/opencode.json → ~/.config/opencode/opencode.json → .well-known/opencode (org defaults).
- **Docs:** opencode.ai/docs · **GitHub:** github.com/anomalyco/opencode

YHK

Resources

- GitHub: `github.com/opencode-ai/opencode`
- Docs: `opencode.ai/docs`
- NXCode.io "OpenCode Tutorial 2026: Install & Setup After Anthropic Block (Updated Guide)"
- JP Caparas, "The definitive guide to OpenCode: from first install to production workflows", Dev Genius / reading.sh, Feb 2026. `blog.devgenius.io`

YHK